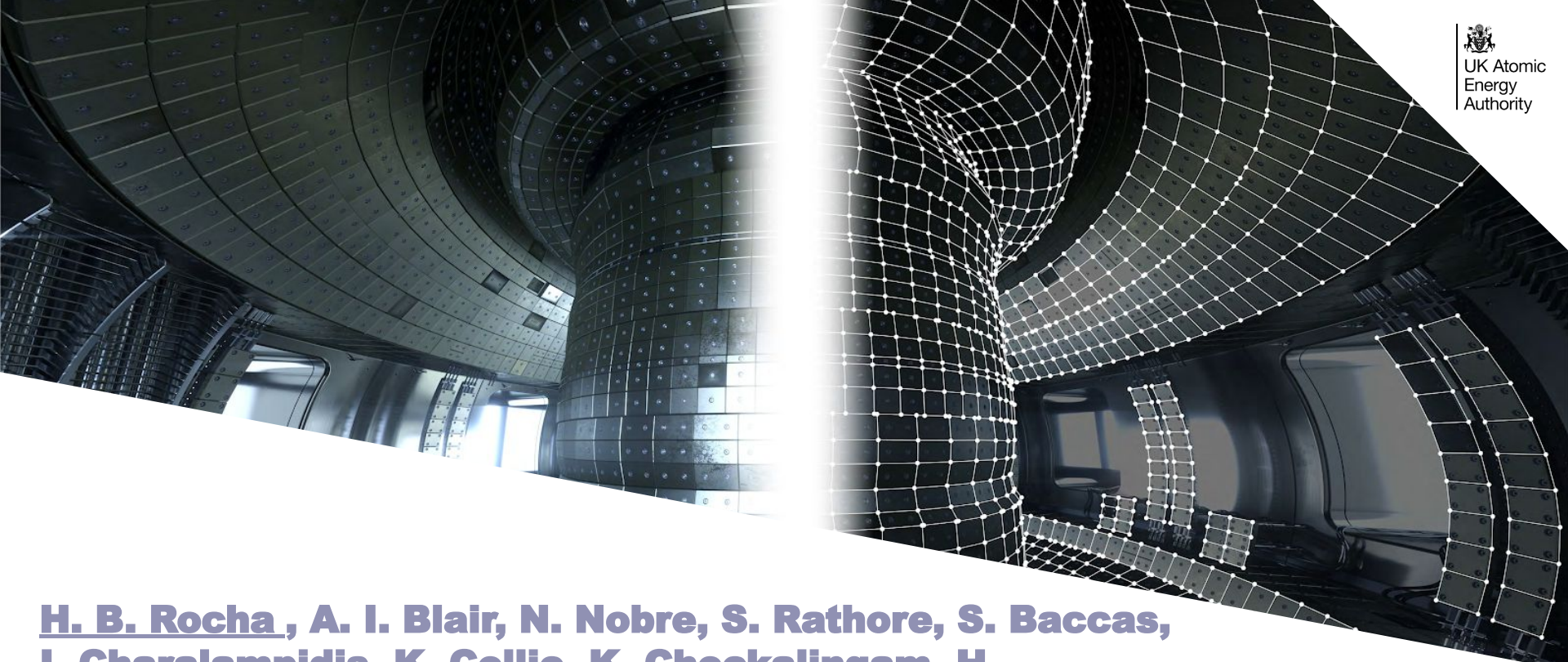




**H. B. Rocha, A. I. Blair, N. Nobre, S. Rathore, S. Baccas,
I. Charalampidis, K. Collie, K. Chockalingam, H.
Saunders, A. Davis**

Composing Problems in MFEM-MOOSE

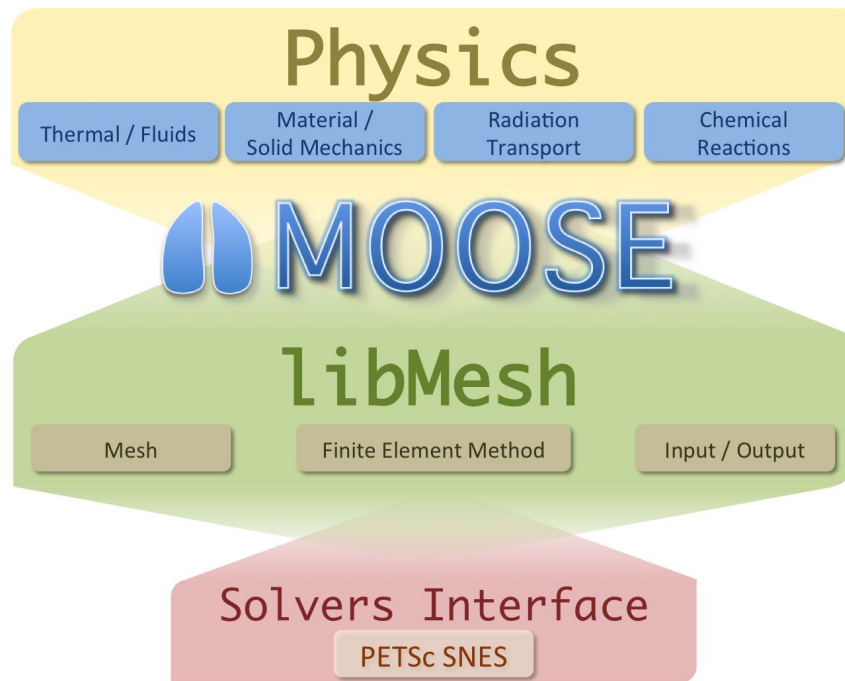
MFEM Community Workshop | September 2026

1. MFEM-MOOSE
2. The ProblemComposer
3. Example: 2D Rogers-Ricci Plasma Model
4. Conclusion

MFEM-MOOSE

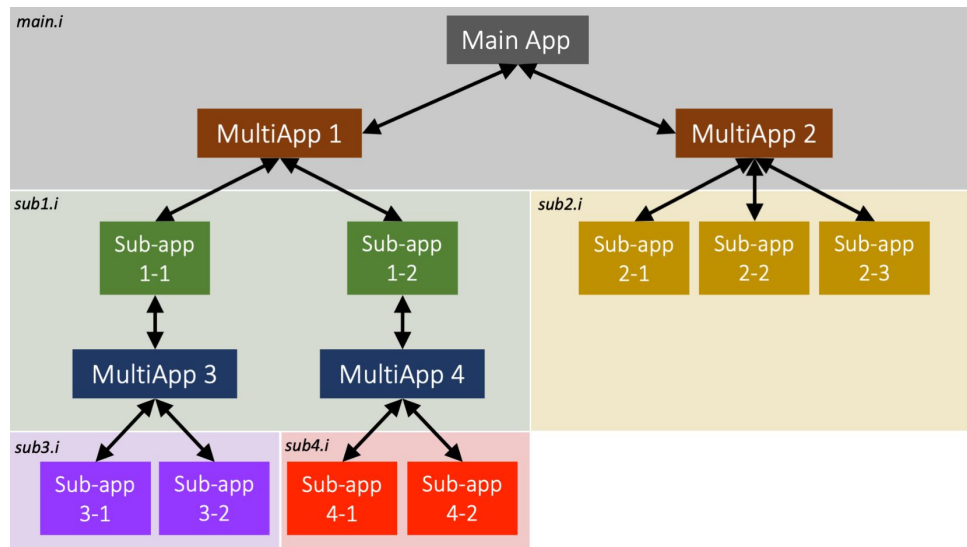
What is MOOSE?

- **Open source** parallel C++ FE framework from Idaho National Laboratory (INL)
- **Large** user community (~2.3k ★ on GitHub)
- **Wide range** of existing validated physics modules
- **Scalable** to $O(10^4)$ CPU cores
- **Simple interface** - no *a priori* coding knowledge required



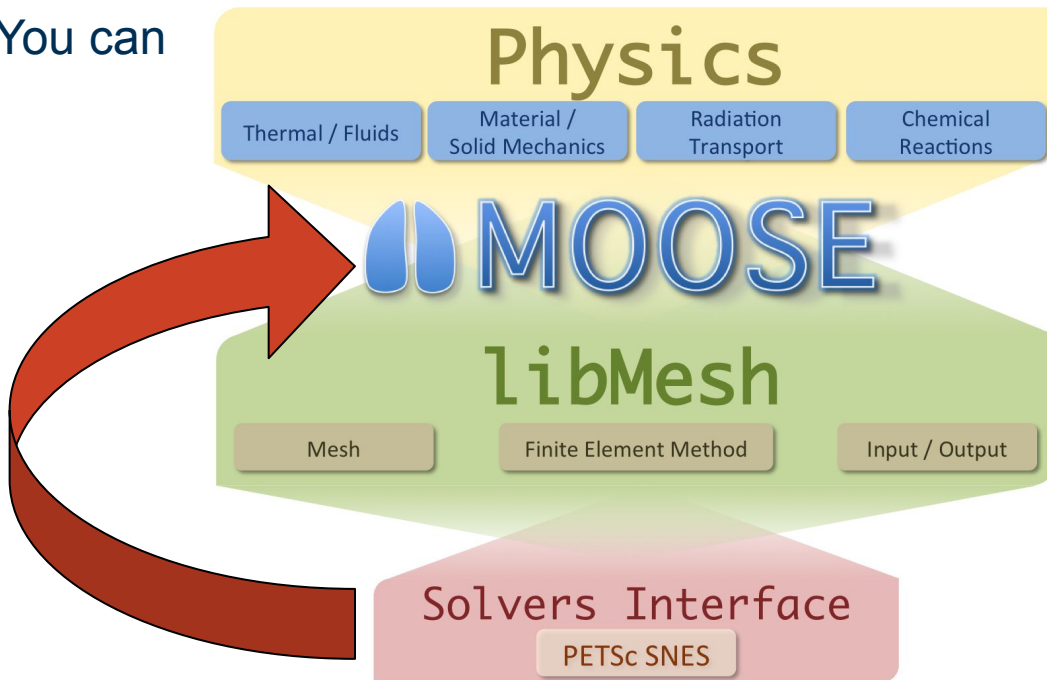
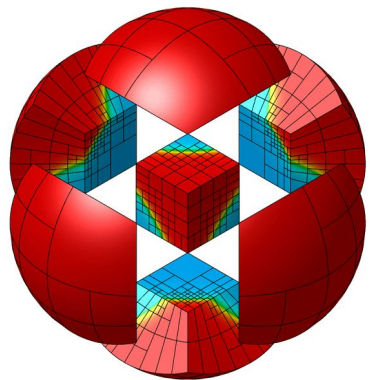
MOOSE transfers

- Of particular note is MOOSE's modularity through app and sub-app transfers.
- Allows for composable multiscale multiphysics.
- Crucial for complex fusion applications.



What is MFEM-MOOSE?

Part of the MOOSE framework. You can use MFEM as the FEM backend instead of libMesh.



How using MFEM-MOOSE usually goes

diffusion.i

```
[Mesh]
  type = MFEMFileMesh
  file = ../mesh/mug.e
[]

[Problem]
  type = MFEMProblem
[]

[FESpaces]
  [H1FESpace]
    type = MFEMScalarFESpace
    fec_type = H1
    fec_order = FIRST
  []
  [HCurlFESpace]
    type = MFEMVectorFESpace
    fec_type = ND
    fec_order = FIRST
  []
[]

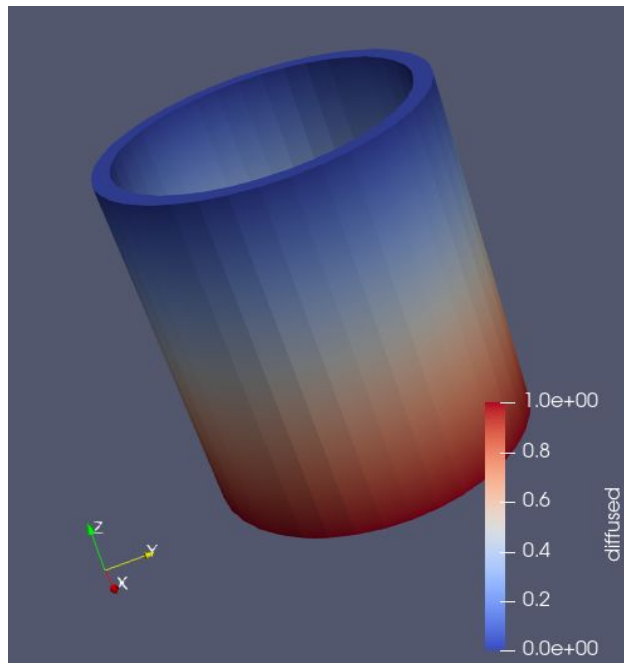
[Variables]
  [concentration]
    type = MFEMVariable
    fespace = H1FESpace
  []
[]
```

```
[BCs]
  [bottom]
    type = MFEMScalarDirichletBC
    variable = concentration
    boundary = 'bottom'
    coefficient = 1.0
  []
  [top]
    type = MFEMScalarDirichletBC
    variable = concentration
    boundary = 'top'
  []
[]

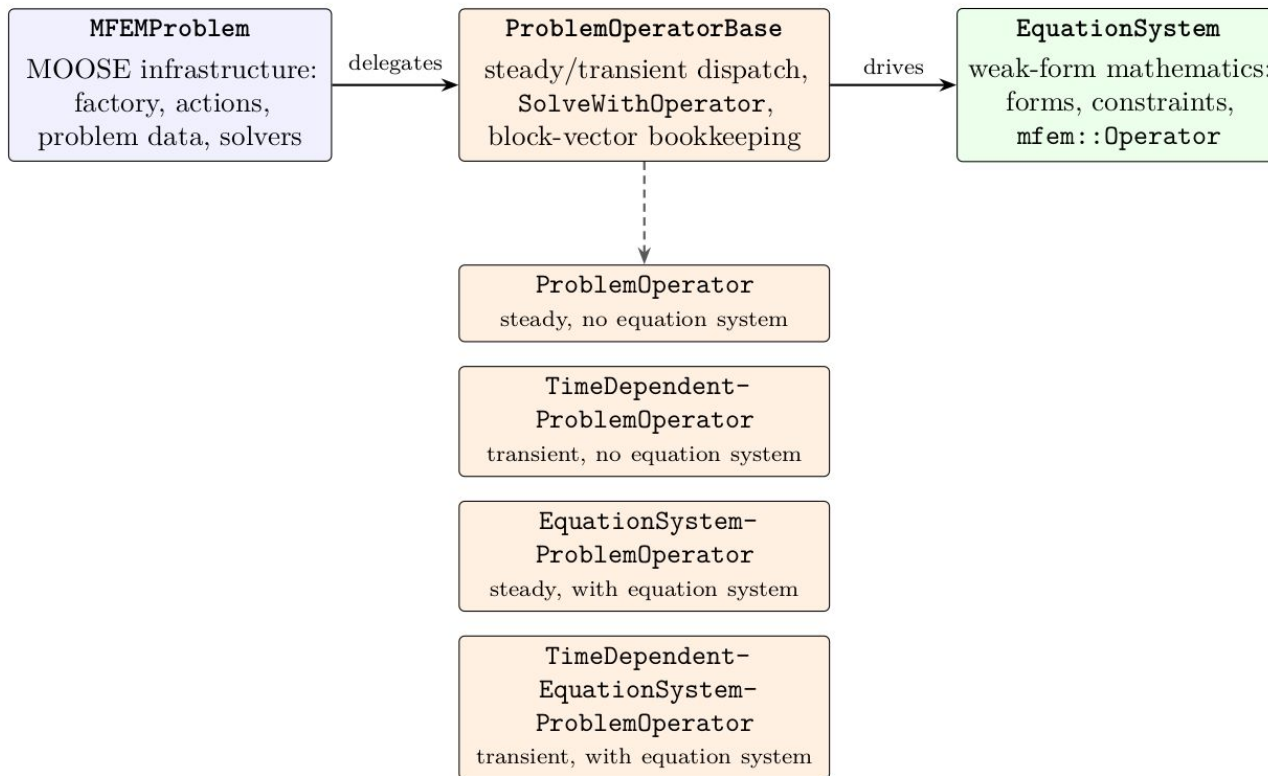
[Kernels]
  [diff]
    type = MFEMDiffusionKernel
    variable = concentration
  []
[]

[Solvers]
  [boomeramg]
    type = MFEMHyprBoomerAMG
  []
  [main]
    type = MFEMHyprGMRES
    preconditioner = boomeramg
    l_tol = 1e-16
    l_max_its = 1000
  []
[]
```

$$\nabla \cdot (\alpha \nabla T) = 0$$



Pipeline



- However, not every MFEM feature has yet been ported to MOOSE.
- Also not every native MFEM problem can be easily cast into MOOSE's structure.
- We need a problem-agnostic way of attaching MFEM-native code to a MOOSE problem such that it can interface with other modules.
- We can now do this by just writing a custom **ProblemComposer** and **ProblemOperator**.

The ProblemComposer

The ProblemComposer

- The **ProblemComposer** is a layer that performs some validity checks (real/complex, steady/transient, eigenproblem, etc).
- It instantiates the correct **ProblemOperator** and directs the executioner to it.
- Here we can use one of the default **ProblemComposer** options or insert a custom one.

Composing a problem

- The executioner selects which problem composer will provide a problem operator

```
// If no ProblemOperators have been added by the user, add a default
if (!_mfem_problem.getProblemComposer())
{
    std::string name = "_DefaultWeakFormProblemComposer";
    InputParameters params = _factory.getValidParams("MFEMWeakFormProblemComposer");

    if (dynamic_cast<MFEMEigenproblem *>(&mfem_problem))
        _mfem_problem.addMFEMProblemComposer("MFEMEigenWeakFormProblemComposer", name, params);
    else if (_mfem_problem.getNumericType() == MFEMProblem::NumericType::COMPLEX)
        _mfem_problem.addMFEMProblemComposer("MFEMComplexWeakFormProblemComposer", name, params);
    else
        _mfem_problem.addMFEMProblemComposer("MFEMWeakFormProblemComposer", name, params);
}
addProblemOperator(_mfem_problem.getProblemComposer()->createProblemOperator(_mfem_problem));
}
```

MFEMSteady.C

- The problem composer creates a problem operator

```
std::shared_ptr<Moose::MFEM::ProblemOperatorBase>
MFEMWeakFormProblemComposer::createProblemOperator(MFEMProblem & mfem_problem)
{
    if (mfem_problem.getNumericType() != MFEMProblem::NumericType::REAL)
        mooseError("Wrong numeric type. Please set the Problem numeric type to 'real'.");

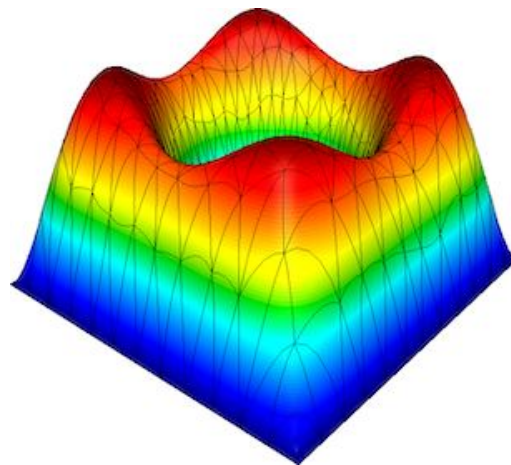
    mfem_problem.getProblemData().eqn_system = std::make_shared<Moose::MFEM::EquationSystem>();
    return std::make_shared<Moose::MFEM::EquationSystemProblemOperator>(mfem_problem);
}
```

MFEMWeakFormProblemComposer.C

Basic Example

MOOSE's main branch has an example test `custom_composer_and_operator.i` which solves a simple Poisson problem (MFEM ex0) using a `CustomProblemComposer` and a `CustomProblemOperator`...

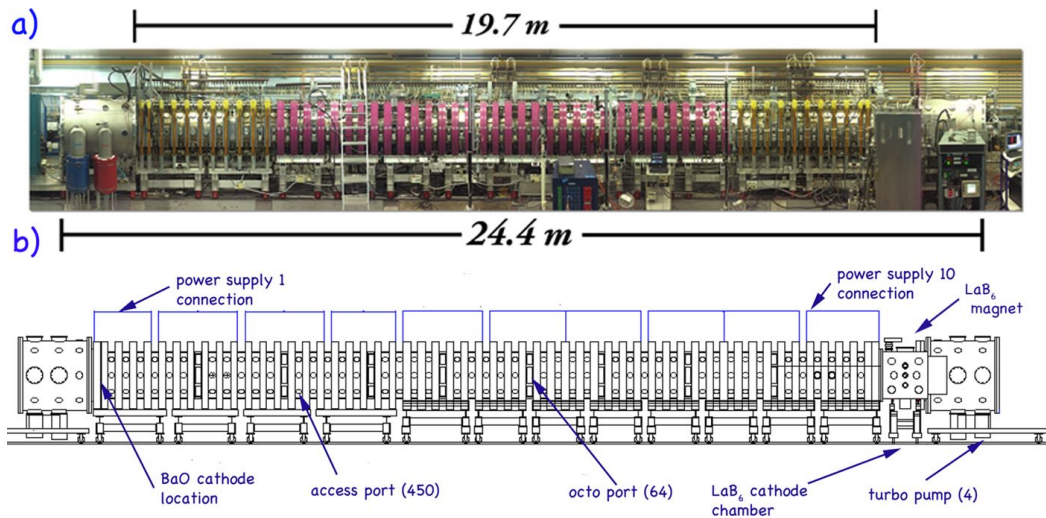
... but let's stress-test our problem composers on something a little more involved



Example: 2D Rogers-Ricci Plasma Model

The Model

- Models the behaviour of linear plasma devices with a parallel magnetic field [1].
- Of specific interest is the LAPD (Large Plasma Device) at UCLA
- $B_{\parallel} = 40\text{--}250\text{ mT}$, $n \sim 10^{12}\text{ cm}^{-3}$



The 2001 version of the LAPD [2]

[1] Rogers, B. N., & Ricci, P. (2010). Low-frequency turbulence in a linear magnetized plasma. *Physical Review Letters*, 104(22), 225002. <https://doi.org/10.1103/PhysRevLett.104.225002>

[2] Gekelman, W., Pribyl, P., Lucky, Z., Drandell, M., Leneman, D., Maggs, J., Vincena, S., Van Compernelle, B., Tripathi, S. K. P., Morales, G., Carter, T. A., Wang, Y., & DeHaas, T. (2016). The upgraded Large Plasma Device, a machine for studying frontier basic plasma physics. *Review of Scientific Instruments*, 87(2), 025105. <https://doi.org/10.1063/1.4941079>

RR2D Strong Form

- Original model in 3D, can be approximately integrated in the parallel direction to yield a 2D model of a square cross-section spanning $(-L/2, L/2)$.
- Time-dependent
- Nonlinear
- 4 variables (reducible to 3):
 - T = temperature
 - n = density
 - ϕ = potential
 - ω = vorticity
- Dirichlet BCs for ϕ , Neumann for everything else.

$$\frac{\partial T}{\partial t} = 40[\phi, T] - \frac{T}{36} \left(1.71e^{3-\phi/T} - 0.71 \right) + S_T$$

$$\frac{\partial n}{\partial t} = 40[\phi, n] - \frac{n}{24} e^{3-\phi/T} + S_n$$

$$\frac{\partial \nabla^2 \phi}{\partial t} = 40 [\phi, \nabla^2 \phi] + \frac{1}{24} \left(1 - e^{3-\phi/T} \right)$$

$$\nabla^2 \phi = \omega$$

where

$$S_n = S_T = 0.03 \left[1 - \tanh \frac{r - r_s}{L_s} \right]$$

Linearly-implicit Euler: each step represents a Newton iteration

$$Mk + K(x + \Delta t k) + b(x + \Delta t k) = 0$$



$$\left(M + \Delta t K + \Delta t \frac{\partial b}{\partial x}\right)k = -(Kx + b(x)) + O(\Delta t^2)$$

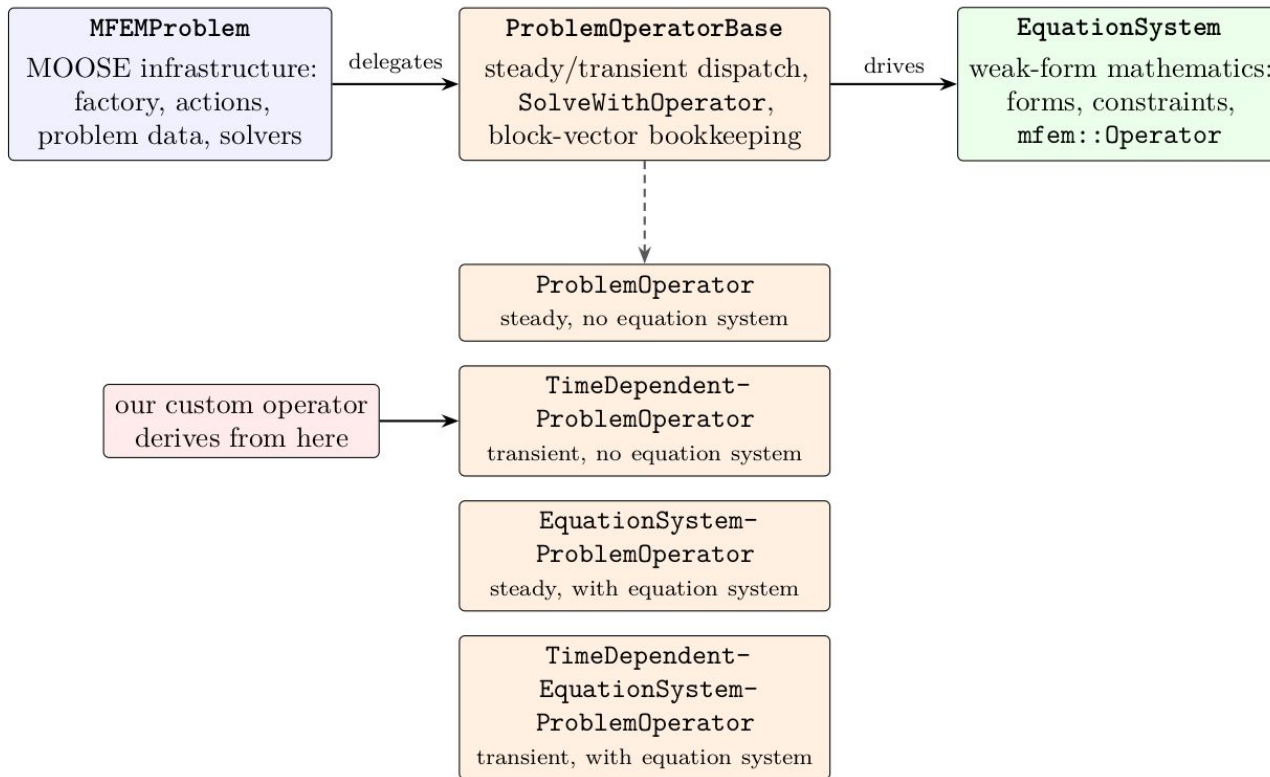
The Block Operator

	T	ω	n	ϕ
T	$M + \Delta t K$ $+ D_T F_T$			$D_\phi F_T$ $+ \Delta t \text{Conv}(R \nabla T)$
ω	$D_T F_\omega$	$M + \Delta t K$		$D_\phi F_\omega$ $+ \Delta t \text{Conv}(R \nabla \omega)$
n	$D_T F_n$		$M + \Delta t K$ $+ D_n F_n$	$D_\phi F_n$ $+ \Delta t \text{Conv}(R \nabla n)$
ϕ		$\Delta t M$		$\Delta t L$

Blank cells are null pointers; `HypreParMatrixFromBlocks` treats them as zero blocks.

- **M** : Mass matrix
- **K** : Poisson bracket + streamline-upwind stabilisation
- **D_iF_j** : Gateaux derivative of reaction term j w.r.t field i
- **L** : Diffusion matrix
- **Conv(R∇T)** : Gateaux derivative of poisson bracket w.r.t ϕ

Where it goes



RogersRicci2DProblemComposer.C

```
InputParameters
RogersRicci2DProblemComposer::validParams()
{
    InputParameters params = MFEMProblemComposer::validParams();
    params.addClassDescription("Composes the problem operator for the 2D Rogers-Ricci "
        "drift-turbulence model.");
    params.addRequiredParam<VariableName>("temperature", "The temperature variable, T.");
    params.addRequiredParam<VariableName>("vorticity", "The vorticity variable, omega.");
    params.addRequiredParam<VariableName>("density", "The density variable, n.");
    params.addRequiredParam<VariableName>("potential",
        "The electrostatic potential variable, phi.");
    params.addRequiredParam<MFEMScalarCoefficientName>("source",
        "Particle source S n. In the normalised equations this also supplies the "
        "temperature source S T.");
    params.addParam<mfem::real_t>("b_inv", 40.0, "Prefactor on the Poisson brackets, 1/B in normalised units.");
    params.addParam<mfem::real_t>("lambda", 3.0, "Constant in the exponent of the sheath-loss terms.");
    params.addParam<mfem::real_t>("eps_squared", 1e-12,
        "Regularisation keeping sqrt(T^2 + eps_squared) bounded away from zero.");
    return params;
}

RogersRicci2DProblemComposer::RogersRicci2DProblemComposer(const InputParameters & parameters)
    : MFEMProblemComposer(parameters)
{
}

std::shared_ptr<Moose::MFEM::ProblemOperatorBase>
RogersRicci2DProblemComposer::createProblemOperator(MFEMProblem & mfem_problem)
{
    // Ordered to match RogersRicciVarIdx, which fixes the block ordering of the system.
    const std::vector<VariableName> variable_names({
        getParam<VariableName>("temperature"),
        getParam<VariableName>("vorticity"),
        getParam<VariableName>("density"),
        getParam<VariableName>("potential")});

    return std::make_shared<RogersRicci2DProblemOperator>({
        mfem_problem,
        variable_names,
        getParam<MFEMScalarCoefficientName>("source"),
        getParam<mfem::real_t>("b_inv"),
        getParam<mfem::real_t>("lambda"),
        getParam<mfem::real_t>("eps_squared")});
}
```

} Wire the **ProblemOperator** to the parameter names given by the user on the script.

} Add the extra tunable parameters which will be passed to the **ProblemOperator**.

} Create the **ProblemOperator**.

RogersRicci2DProblemOperator.h

```
class RogersRicci2DProblemOperator : public Moose::MFEM::TimeDependentProblemOperator
{
public:
    RogersRicci2DProblemOperator(MFEMProblem & problem,
                                const std::vector<VariableName> & variable_names,
                                const MFEMScalarCoefficientName & source_name,
                                mfem::real_t b_inv,
                                mfem::real_t lambda,
                                mfem::real_t eps_squared);

    ~RogersRicci2DProblemOperator() override;

    virtual void SetGridFunctions() override;
    virtual void Init(mfem::BlockVector & X) override;
    virtual void Solve() override;
    virtual void
    ImplicitSolve(const mfem::real_t dt, const mfem::Vector & x, mfem::Vector & k) override;

protected:
    void declareCoefficients();
    void formSystem(mfem::real_t dt);
    void deleteBlocks();

private:
    const std::vector<VariableName> _variable_names;
    const MFEMScalarCoefficientName _source_name;
    const mfem::real_t _b_inv;
    const mfem::real_t _lambda;
    const mfem::real_t _eps_squared;
    mfem::real_t _dt{0.0};
    mfem::ParFiniteElementSpace * _fespace{nullptr};
    mfem::real_t _element_size{0.0};
    mfem::Array<int> _phi_ess_tdofs;
    mfem::BlockVector * _state{nullptr};
    mfem::Array2D<const mfem::HypreParMatrix *> _h_blocks;
    std::unique_ptr<mfem::HypreParMatrix> _system_operator;
};
```

Updates variables and calls the ODE Solver step

Calls `formSystem()` and `SolveWithOperator()` and passes the system operator to it.

Adds all the needed coefficients to MFEM-MOOSE's **CoefficientManager**. Sets coefficients for the reaction terms, the Gateaux derivatives, etc.

Forms the block matrix and sets it as the system operator. This is where most of the code goes.

A Snippet of formSystem()

```
// K = E x B advection (the Poisson bracket) + streamline-upwind stabilisation.
auto * k_int = new mfem::SumIntegrator;
k_int->AddIntegrator(new mfem::ConvectionIntegrator(drift, -_b_inv));
k_int->AddIntegrator(new mfem::DiffusionIntegrator(suw));
// ScaleIntegrator does not take ownership here: k owns k_int exclusively.
auto * k_dt_int = new Moose::MFEM::ScaleIntegrator(k_int, dt, false);

k.AddDomainIntegrator(k_int);
k.Assemble();
k.Finalize();

auto * m_int = new mfem::SumIntegrator;
m_int->AddIntegrator(new mfem::MassIntegrator); // Time derivative term
if (i == RR_T)
  m_int->AddIntegrator(
    new mfem::MassIntegrator(coefs.getScalarCoefficient(coefName("DT_FT"))));
else if (i == RR_N)
  m_int->AddIntegrator(
    new mfem::MassIntegrator(coefs.getScalarCoefficient(coefName("Dn_Fn"))));
// RR_OMEGA has no diagonal Gateaux term.
m_int->AddIntegrator(k_dt_int);

m.AddDomainIntegrator(m_int);
m.Assemble();
m.Finalize();

// RHS = -K x - (F_i, v). ParallelAssemble sums local contributions across ranks
// (P^T b) and TrueAddMult applies the true-DoF action P^T K P.
b.AddDomainIntegrator(
  new mfem::DomainLFIntegrator(coefs.getScalarCoefficient(reactionCoefName(i))));
b.Assemble();
std::unique_ptr<mfem::HypreParVector> b_true(b.ParallelAssemble());
*b_true *= -1.0;
k.TrueAddMult(state.GetBlock(i), *b_true, -1.0);
```

And finally the script

rogers_ricci_2d.i

```
[Mesh]
  type = MFEMFileMesh
  file = ../mesh/rogers_ricci_64x64.mesh
  uniform_refine = 2
[]

[Problem]
  type = MFEMProblem
[]

[FESpaces]
  [H1FESpace]
    type = MFEMScalarFESpace
    fec_type = H1
    fec_order = FIRST
  []
[]

[Variables]
  [T]
    type = MFEMVariable
    fespace = H1FESpace
  []
  [omega]
    type = MFEMVariable
    fespace = H1FESpace
  []
  [n]
    type = MFEMVariable
    fespace = H1FESpace
  []
  [phi]
    type = MFEMVariable
    fespace = H1FESpace
  []
[]

[Functions]
  [Sn]
    type = ParsedFunction
    expression = '0.5 * S_0n * (1 - tanh((sqrt((x - xL / 2)^2 + (y - yL / 2)^2) - r_s) / L_s))'
    symbol_names = 'S_0n xL yL r_s L_s'
    symbol_values = '0.03 100.0 100.0 20.0 0.5'
  []
[]
```

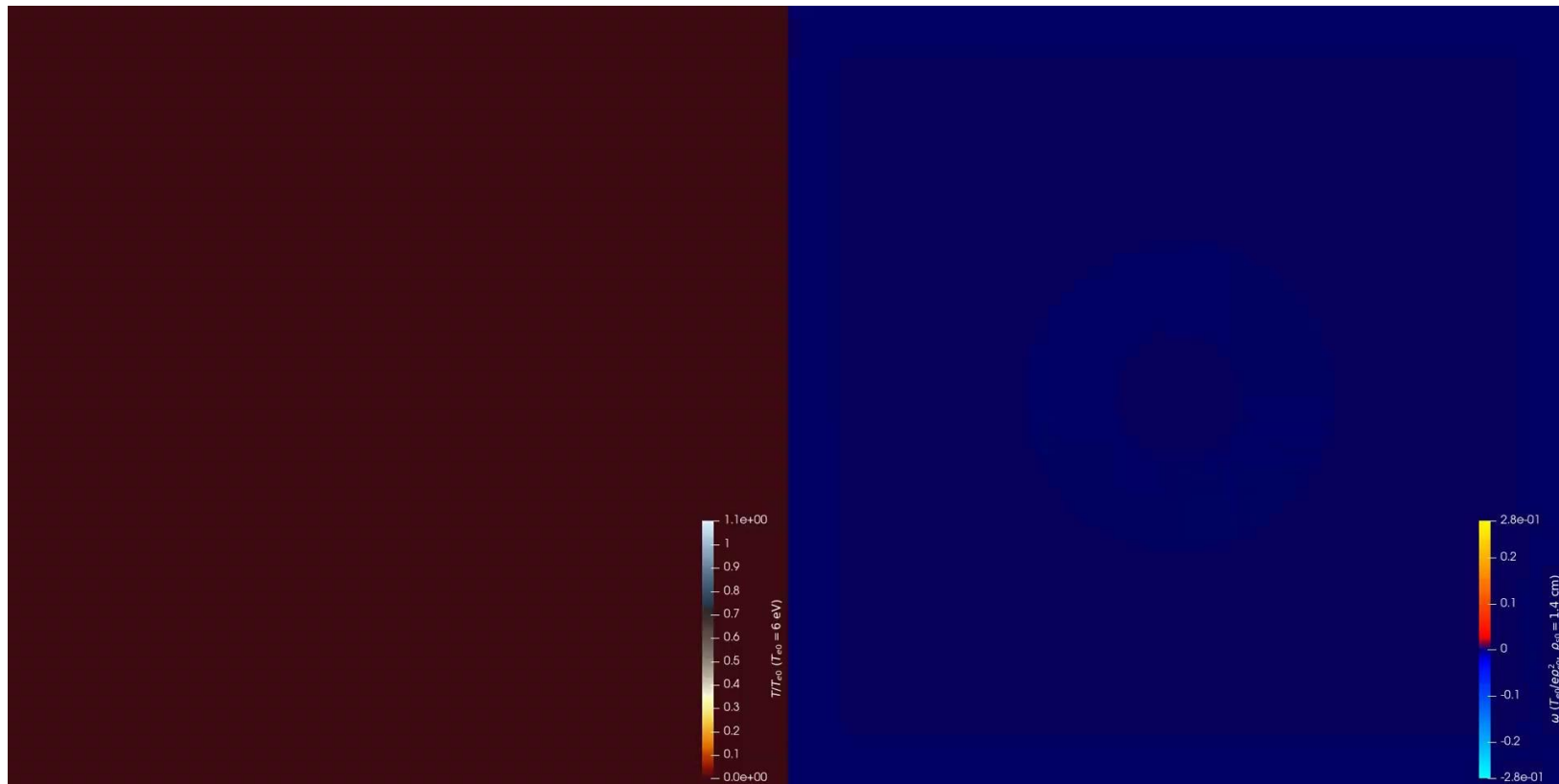
```
[ICs]
  [T_ic]
    type = MFEMScalarIC
    variable = T
    coefficient = 1e-4
  []
  [omega_ic]
    type = MFEMScalarIC
    variable = omega
    coefficient = 0.0
  []
  [n_ic]
    type = MFEMScalarIC
    variable = n
    coefficient = 1e-4
  []
  [phi_ic]
    type = MFEMScalarIC
    variable = phi
    coefficient = 0.03
  []
[]

[ProblemComposers]
  [rogers_ricci]
    type = RogersRicci2DProblemComposer
    temperature = T
    vorticity = omega
    density = n
    potential = phi
    source = Sn
    lambda = 3.0
    b_inv = 40.0
    eps_squared = 1e-4
  []
[]

[Solvers]
  [superlu]
    type = MFEMSuperLU
  []
[]
```

```
[Executioner]
  type = MFEMTransient
  device = cpu
  dt = 0.12
  start_time = 0.0
  end_time = 120
[]

[Outputs]
  [ParaViewDataCollection]
    type = MFEMParaViewDataCollection
    file_base = OutputData/RogersRicci2D_R2
    vtk_format = ASCII
  []
[]
```

T ω 

Conclusion

Conclusion

- MFEM-MOOSE's new problem composer feature allows users to attach MFEM problems to the larger MOOSE framework.
- Even somewhat complicated transient, nonlinear problems may be ported from native MFEM.
- Most of the work is copying and pasting the bulk of the code into a method that forms the system operator.
- The minimal CustomProblemOperator example is already on upstream MOOSE, under `moose/test/tests/mfem/problemcomposers`.
- You can find the Rogers-Ricci example in **Heinrich-BR/moose/HenriqueBR/RR2D_problemcomposer**. QR code on next slide



Thank you!

This work has been funded by the Fusion Futures Programme. As announced by the UK Government in October 2023, Fusion Futures aims to provide holistic support for the development of the fusion sector.